MODULE 06 · AMBITIOUS ENTREPRENEURSHIP

Prototyping with AI

What is the fastest thing you can build to learn?

COURSE PATH

00 01 02 03 04 05 06 07 08 09 10 11 12

CHECK-IN · SLIDO OR HANDS

Did a dummy pass your line?

1. A dummy passed
2. Every dummy fell under the line
3. No line written

2:00

WEEKLY ACCOUNTABILITY

Three questions, every week

1

User interactions?

What did you learn?

2

New revenue?

Why did they pay?

3

Shipped?

Why does it matter?

🦄 **Report target vs. actual in small groups**

CHAPTER 1 OF 9

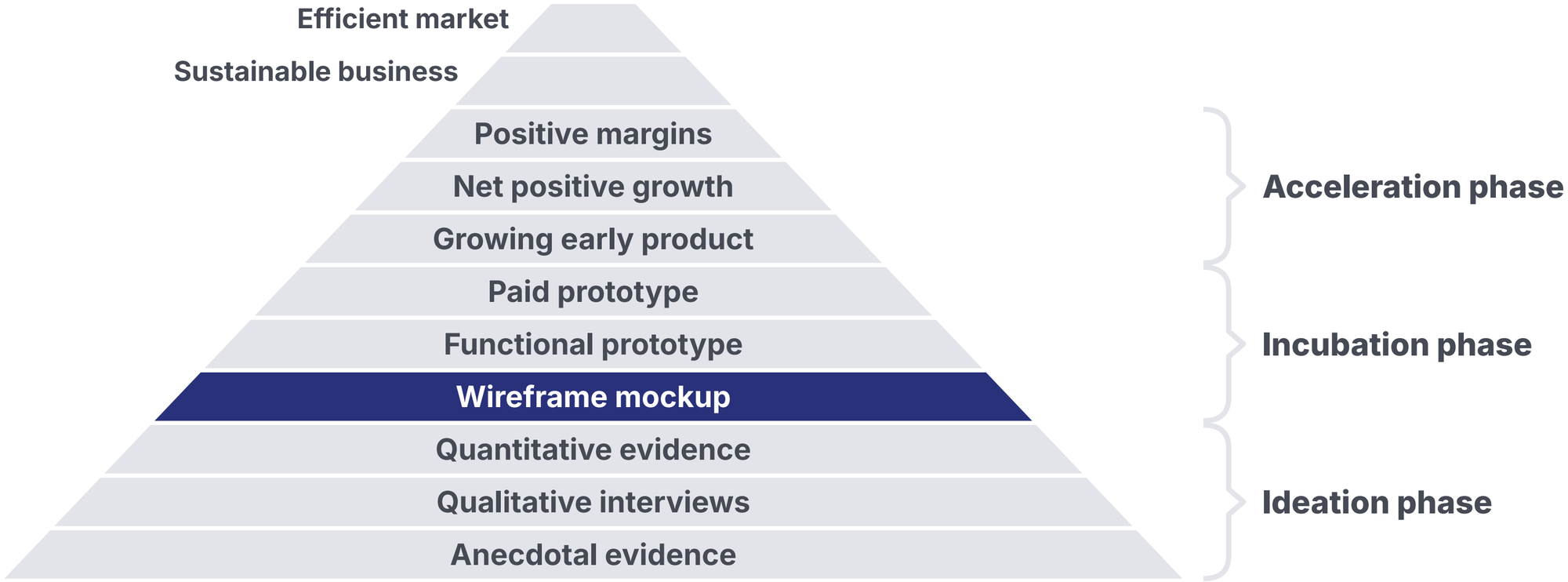
Recap from last time

Where last week left you, a dummy people clicked and asks they accepted.

01 **Recap from last time**
02 Your presentations
03 Every application ever
04 The cheapest version that works
05 You are the product owner
06 Build to get users
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

RECAP · MODULE 05

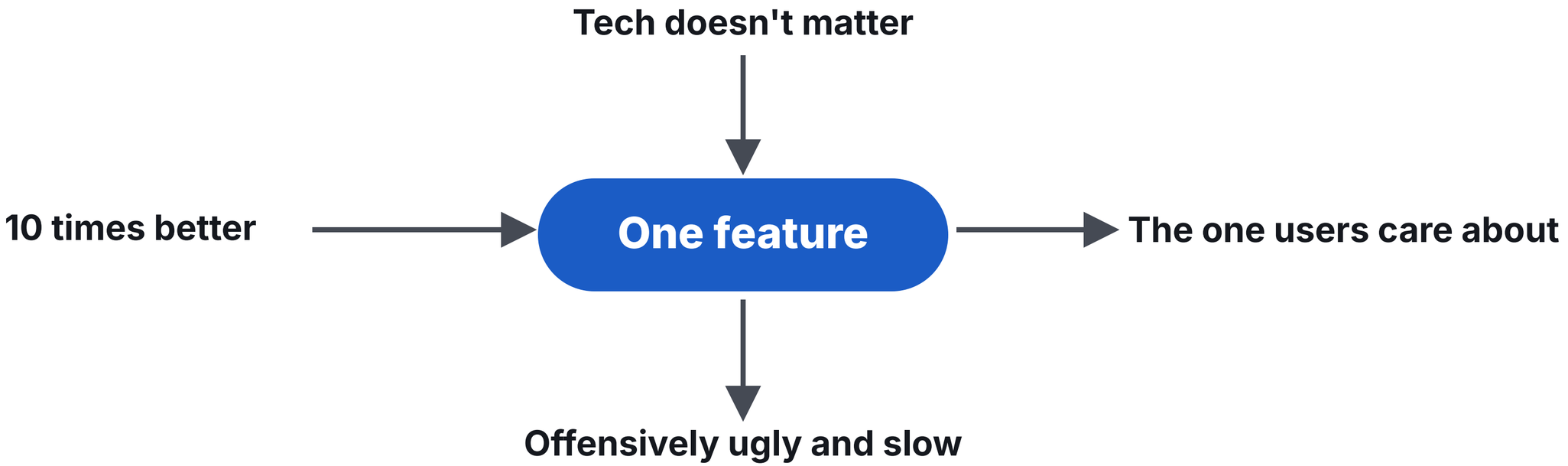
Last week you climbed to the mock-up



RECAP · MODULE 05

You cut it to one feature

The MVP is the smallest thing someone could pay for



RECAP · MODULE 05

Now we are building to get users

Build to find a seed
several dummies, going broad

▶

Build to improve
one winner, going narrow

▶

Build to get users
today

Modules 05 and 06

TODAY'S QUESTION · HANDS

What is the fastest thing you can build to learn?

1

2

1:00

CHAPTER 2 OF 9

Your presentations

Your Module 05 line next to the asks people accepted, three minutes per team.

01 ~~Recap from last time~~
02 **Your presentations**
03 Every application ever
04 The cheapest version that works
05 You are the product owner
06 Build to get users
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

YOUR PRESENTATIONS · MAKE IT CREDIBLE

Show us where you're at in 3 minutes or less!

3:00

Facts

What the segment uses today and what it costs, with its source

Numbers

Guided demos per student, demos per dummy, asks accepted per dummy next to the line

Quotes

Two things people said while clicking, with roles, not names

Images

The winning dummy's key screen and the ask screen

Verdict: which dummy passed, which dropped (type 1), or back to the problem (type 2)

What did you do?

What did you learn? Where did the AI second opinion disagree?

How can we help?

BASTIAN Course check-in format, legacy slides 115 and 387. Categories from the Module 05 workshop.

CHAPTER 3 OF 9

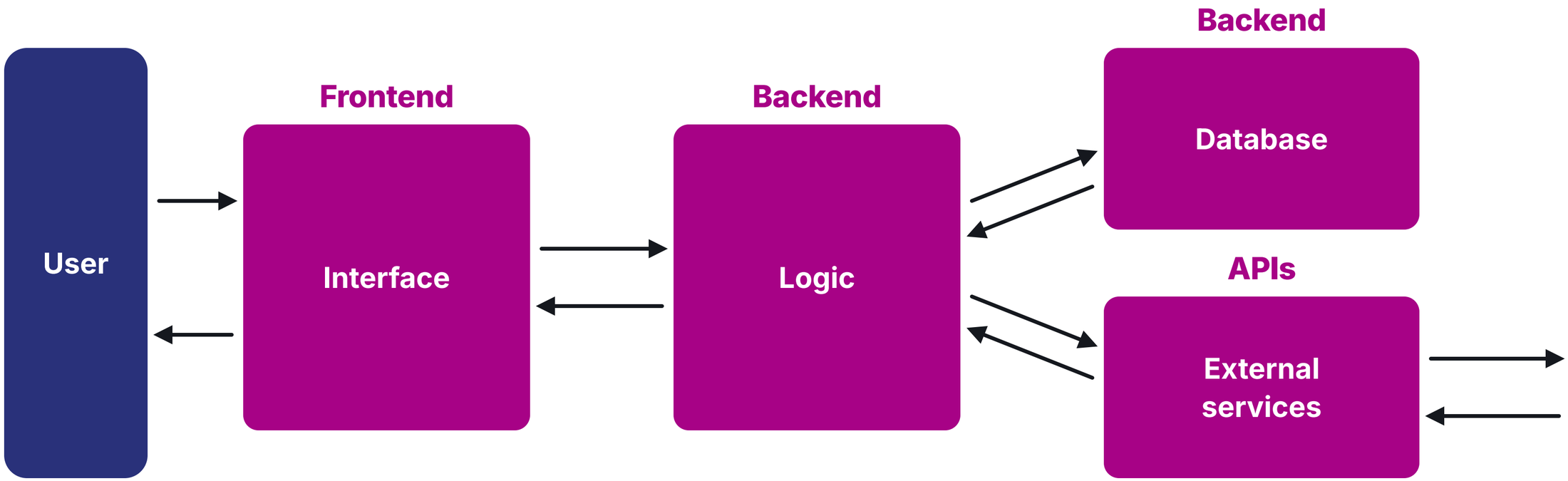
Let's jump in! Every application ever

Every app has the same four parts, so draw yours before anyone builds it.

01 ~~Recap from last time~~
02 ~~Your presentations~~
03 **Every application ever**
04 The cheapest version that works
05 You are the product owner
06 Build to get users
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

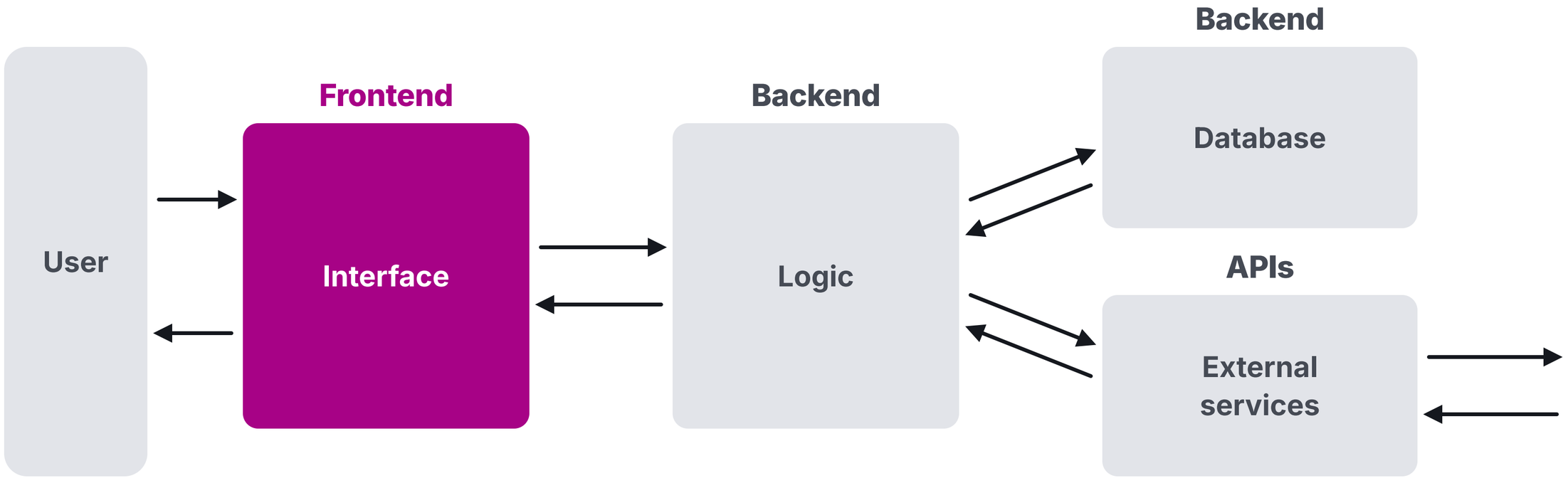
YOUR SHORTCUT TO THINKING LIKE AN ARCHITECT

Every application has the same four parts



EVERY APPLICATION EVER · INTERFACE

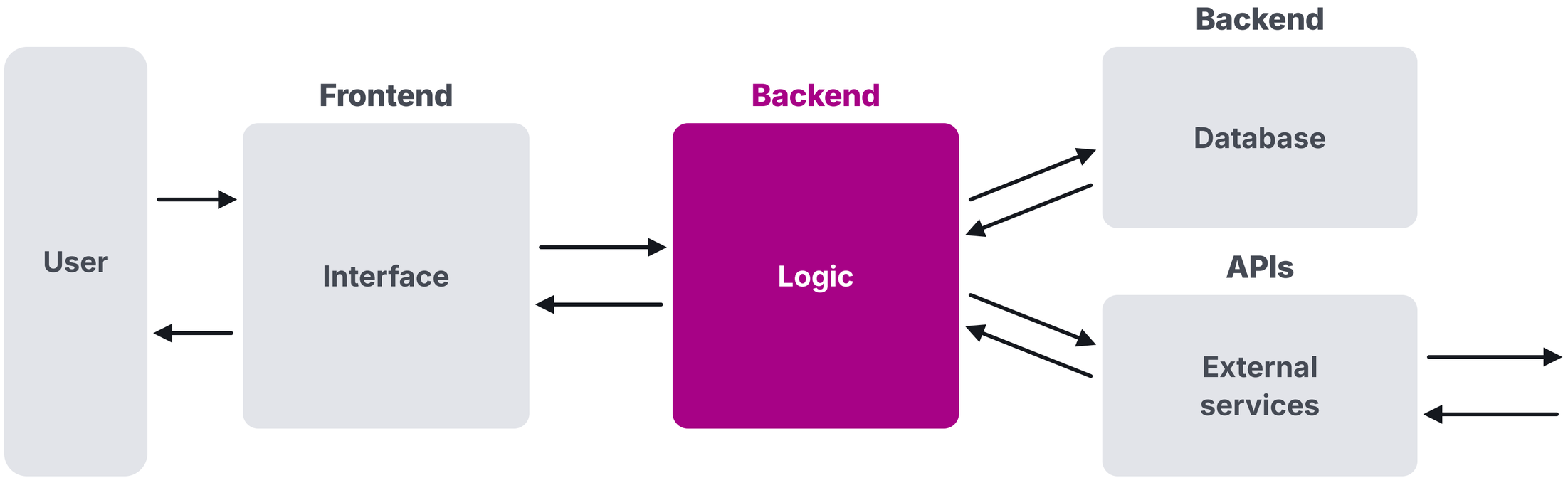
The interface asks for the input



What do you need from the user, and they from you?

EVERY APPLICATION EVER · LOGIC

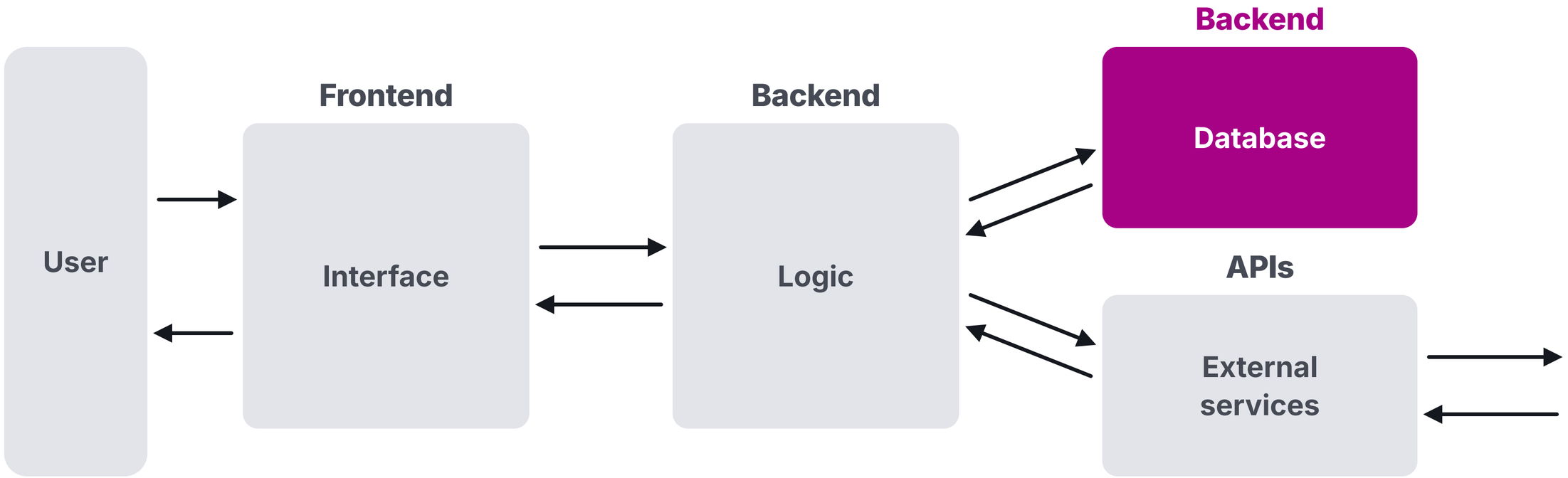
The logic turns input into value



What goes in, what comes out, and what does the user value?

EVERY APPLICATION EVER · DATABASE

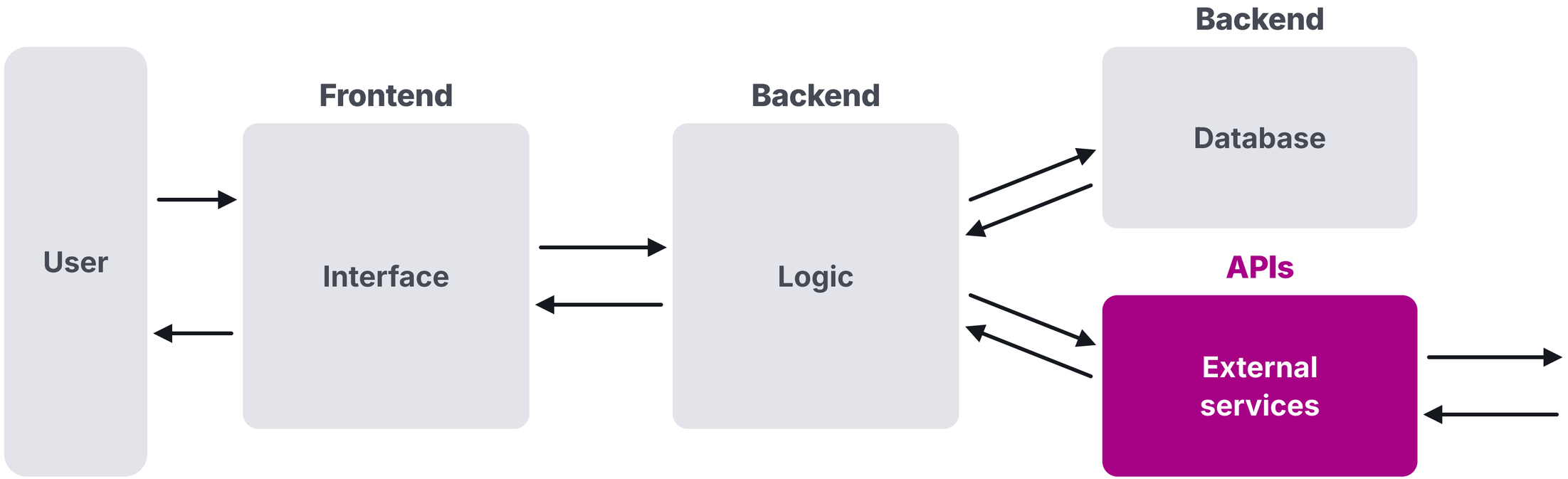
The data is the one part you can't rebuild



How do you protect it from being corrupted?

EVERY APPLICATION EVER · APIS

Outside services come in through APIs



What does a call cost? Set a budget before the first call.

GATE QUESTION · HANDS

Which part did your Module 05 dummy **fake**?

1:00

1. Interface
2. Logic
3. Database
4. External services

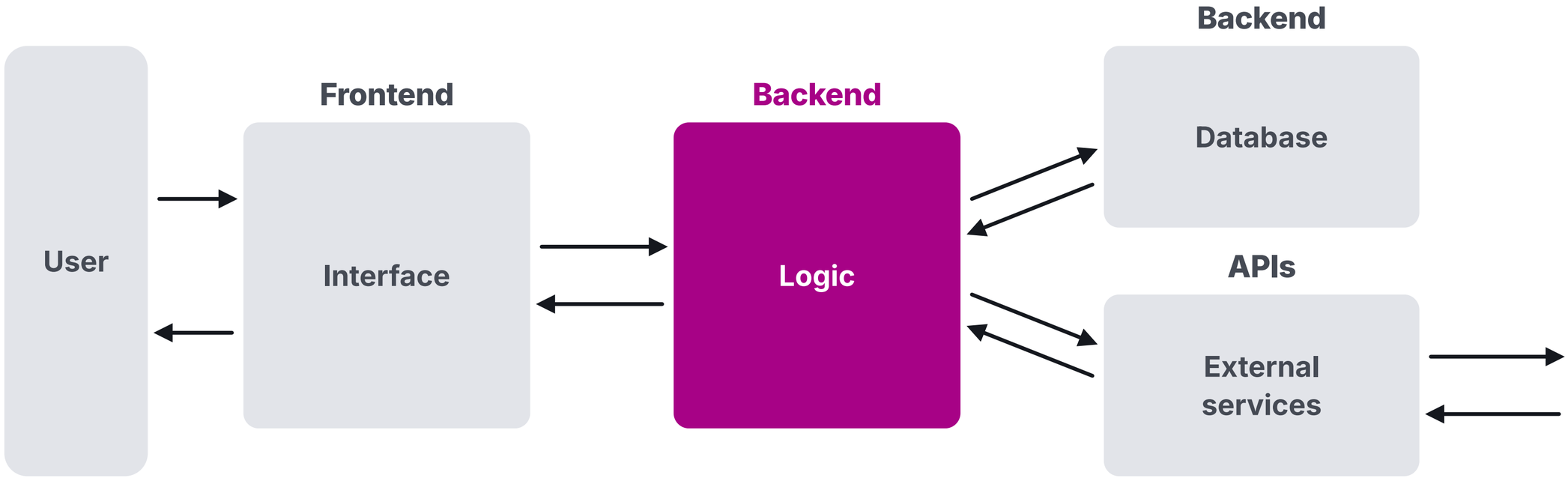
WORKED EXAMPLE

An AI chatbot is the same four parts



WORKED EXAMPLE · QUACKSYNC, MADE UP

Mark what must work, and what you do by hand



Interface
a form and a list

Logic
title and next step, **by hand** if the call fails

Database
one CSV sheet

External services
an AI model

ILLUSTRATIVE QuackSync, the course's made-up venture (legacy slides 407 to 411), mapped on legacy slide 400. The build is on the slides of chapter 5.

PROTOTYPE THE PROCESS FIRST

Lay out the kitchen before you build it

Run the process by hand until it works, then automate the part that hurts

Run it by hand until it works. Then automate the part that hurts.

BASTIAN Legacy slide 415. The tennis-court layout after Hess, A. (2013), Alan Hess on Architecture. The image is a still from The Founder (2016).

CORE PRINCIPLE

Rebuild any part, as often as you like. Lose

0 rows they entered

The code is disposable. What users entered is not.

Module 07 reads who came back.

NOW YOU · ALONE, THEN COMPARE

Draw your MVP on the four parts

Interface	Where does the input go in?
Logic	What turns it into the outcome?
Data	What must you keep?
APIs	Which outside service?
By hand	Which part do you do yourself?

3:00

OUTPUT

Four boxes, one tagged by hand

CHAPTER 4 OF 9

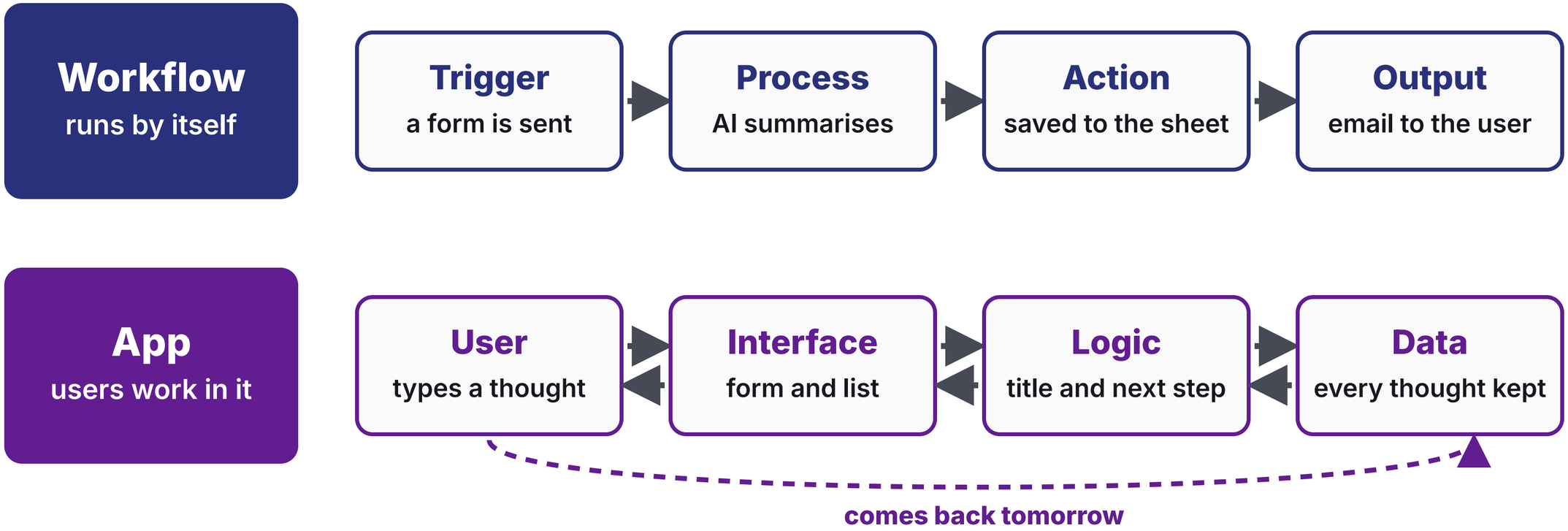
The cheapest version that works

Pick the cheapest form that delivers the outcome, often a form, a sheet and you.

01 ~~Recap from last time~~
02 ~~Your presentations~~
03 ~~Every application ever~~
04 **The cheapest version that works**
05 You are the product owner
06 Build to get users
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

WORKFLOW OR APP

A workflow runs by itself. In an app, users work



BASTIAN Workflow steps from legacy slides 418 to 420 (trigger, action, output) and 457 (input, process, output). The app lane is QuackSync, made up.

GATE QUESTION · HANDS

Workflow or app?

1. A weekly report of new tenders, sent by email
2. A planner where a coach builds a training week
3. An alert when a finalist goes quiet

1:00

THE CHEAPEST VERSION

Who catches the input? You, a tool, or code

No code You catch it	Low code A tool catches it	Code Your code catches it
An email or a message to you	A workflow collects it via API	A frontend built from scratch
A form whose answers you read	A website you build quickly	A native app for your users
A click dummy, you behind it	A chat interface on a server	Built with an AI coding agent

PROTOTYPE WITH A SPREADSHEET

A sheet is your first database

One row per thing, one column per field, fixable by hand

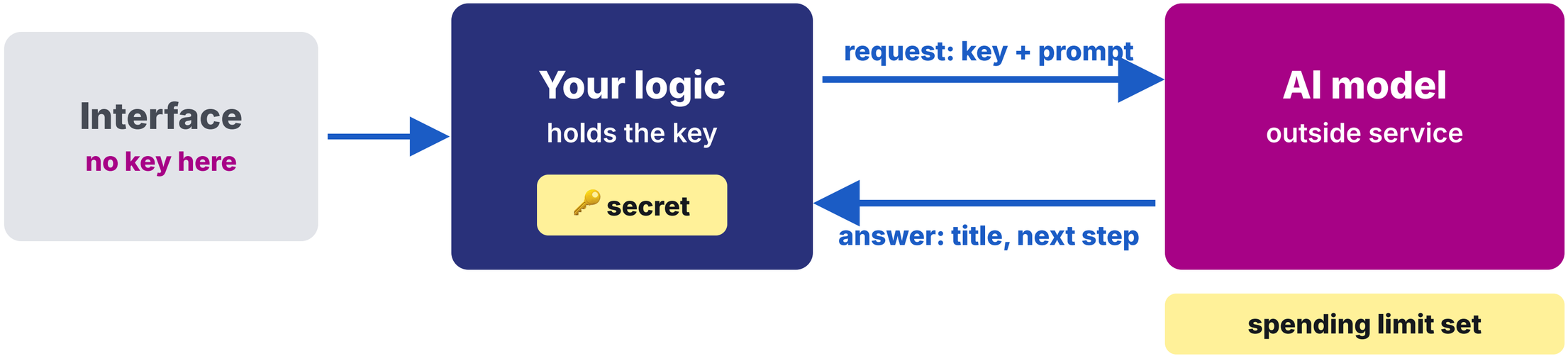
	A · saved	B · thought	C · title	D · next step
2	19 Oct 07:12	what if the bathroom mirror showed my calendar	Mirror calendar	Sketch it on paper
3	19 Oct 07:40	a podcast about failed inventions	Failed inventions podcast	List 5 episodes
4	20 Oct 06:55	duck that reminds me to drink water	(waiting for review)	

When the pieces start to relate to each other, a sheet becomes a database. Until then, a sheet is enough.

ILLUSTRATIVE QuackSync sheet, made up. "DB != Excel", legacy slide 434. Tool cards for Google Sheets and Airtable, legacy slides 26 to 29.

ONE API CALL

Your logic calls the model and keeps the key



BASTIAN Create an API key, set a budget, you see the key only once, legacy slides 424 to 427, redrawn without the tool.

TIP · BEFORE THE FIRST CALL

Set a spending limit, then hide the key

Anyone who sees the key can spend your money

FROM

The key pasted into the page, no limit at the provider.

TO

A limit set first, the key in the builder's secrets or an environment variable.

> You see the key only once. Store it like a password.

BASTIAN Legacy slide 425, "Make sure you have a budget" and "You'll see the key only once!".

THE TOOL FITS THE JOB

If all you have is a hammer, everything looks like a nail

An AI builder makes everything look like an app

Ask first: would a form, a sheet and you deliver the outcome?

BASTIAN Legacy slide 412, "If all you have is a Lovable, everything looks like a SaaS software".

NOW YOU · IN YOUR TEAM

What is the cheapest version that delivers?

Workflow? Does the user only need the result?

App? Does the user work in it and come back?

Catch You, a tool or code?

Sheet Is a sheet enough for the data?

2:00

OUTPUT

One sentence: form, sheet, call, by hand

CHAPTER 5 OF 9

You are the product owner

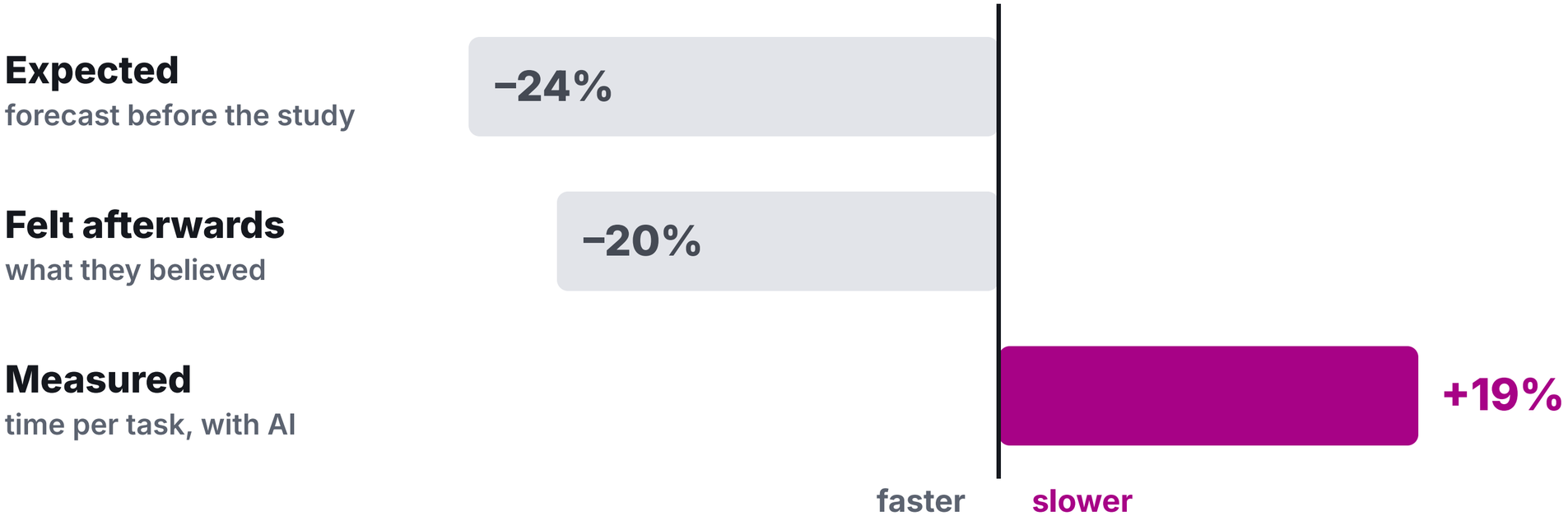
An AI coding agent builds whatever you ask, so ask for small steps and check each one.

01 ~~Recap from last time~~
02 ~~Your presentations~~
03 ~~Every application ever~~
04 ~~The cheapest version that works~~
05 **You are the product owner**
06 Build to get users
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

WHY IT MATTERS

It feels faster. Measure it

16 experienced developers, 246 real tasks, early 2025

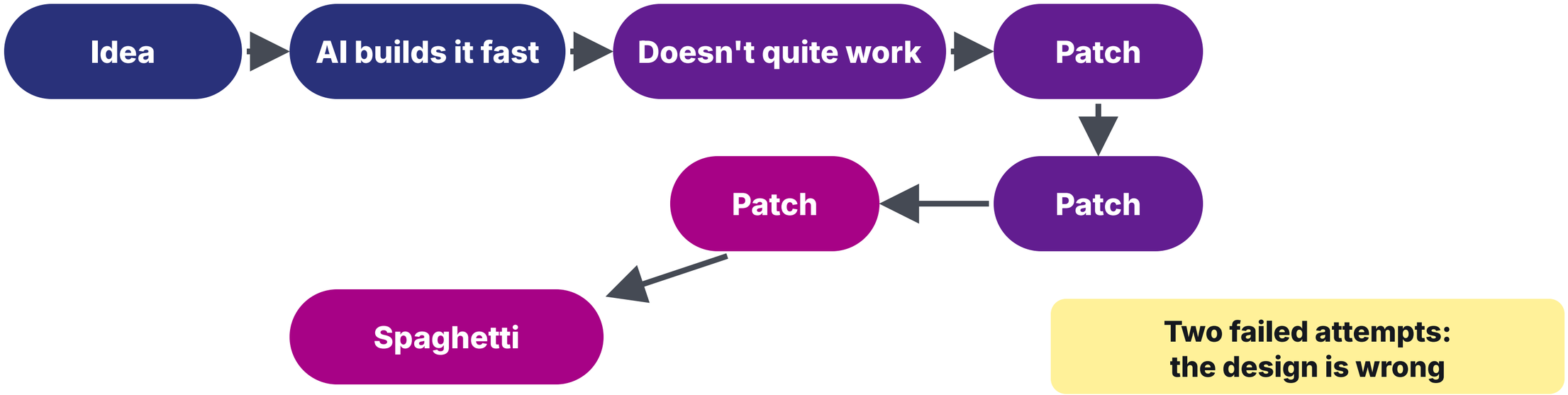


RESEARCH METR (2025), Measuring the impact of early-2025 AI on experienced open-source developer productivity, randomized controlled trial, 10 July 2025.

THE PATCH SPIRAL

Patch, patch, patch, spaghetti

AI removed the bottleneck of building, not the bottleneck of deciding



GATE QUESTION · HANDS

Who had an AI fix one thing and **break** another?

1

2

1:00

CORE PRINCIPLE

You are the product owner. The agent is the contractor

It builds whatever you ask, and it never pushes back on scope

FROM

"Build me a recruiting platform." The agent decides what goes in.

TO

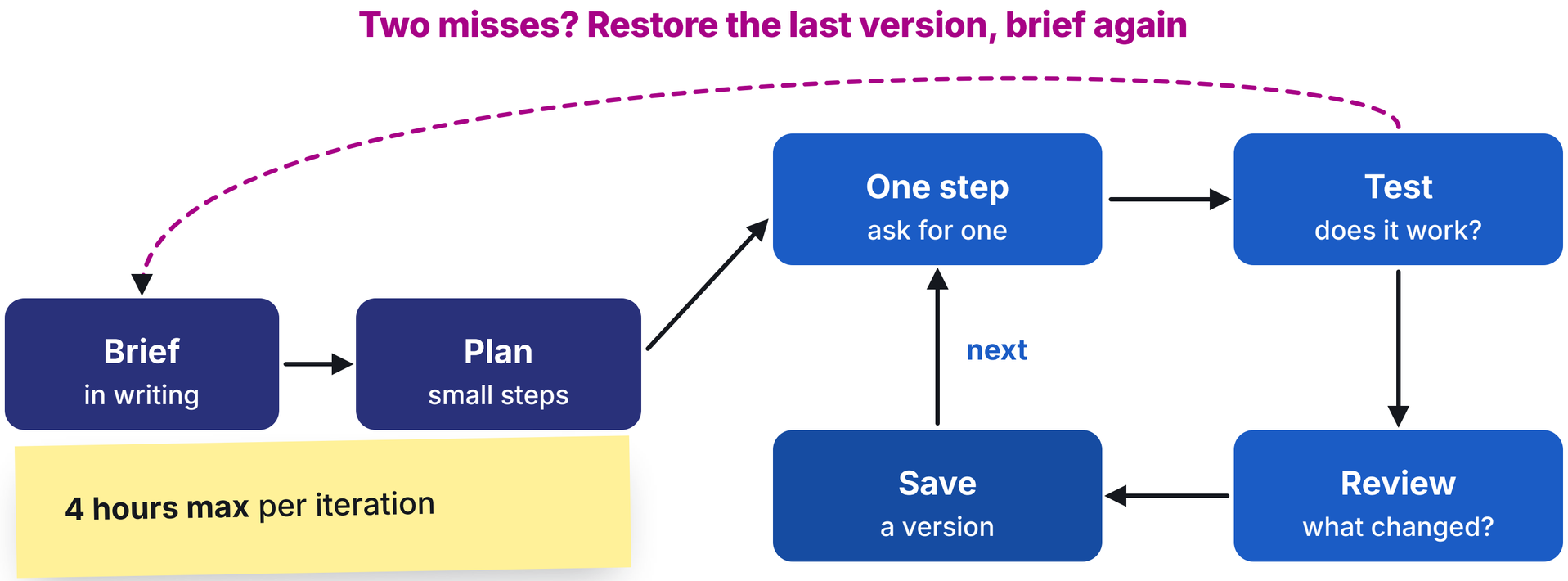
You decide what and why. The agent proposes how. You check it.

> Never let the contractor decide what to build.

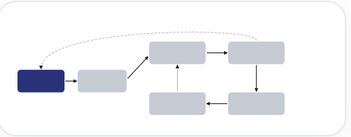
FRAMEWORK Best Practices: Coding with AI, course resource, anti-pattern 5 "The AI Yes-Man Problem" ("You are the PM").

THE AGENT LOOP

Brief, one step, test, review, save



FRAMEWORK Claude Code docs, "Best practices for Claude Code" (explore, plan, implement, commit, and after two failed corrections start fresh with a better prompt), accessed 2026-10-19. Two-miss rule also in the course resource Best Practices: Coding with AI. Legacy slides 377, 448 and 450.



SHOW THE REAL THING · QUACKSYNC · DEMO 1 OF 6 · BRIEF

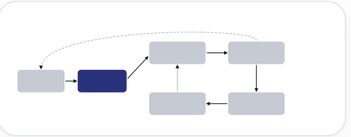
Write the brief once

Who, the one feature, the four parts, what stays manual, what not to build.

PROJECT_INSTRUCTIONS.md

```
# QuackSync · project instructions
## Who it is for
People who get their best ideas in the shower and have
lost them by the time they are dressed.
## The one feature
Type a raw thought into one box. QuackSync saves it and
gives back a one-line title and one next step. All saved
thoughts show in a list, newest first.
## Input and outcome
- Input: free text, up to 500 characters.
- Outcome: the thought is saved with its title and next
step, and the user sees it in the list.
## The parts
- Interface: one web page, a form and the list. Nothing
else.
- Logic: one call to an AI model that returns the title
and the next step.
- Database: a CSV file, `thoughts.csv`, so it opens in any
spreadsheet.
- Outside service: the AI model's API. The key comes from
the environment variable `ANTHROPIC_API_KEY` and never
appears in the page or in the repository.
## What stays manual
If there is no key or the call fails, save the thought
anyway with the title "(waiting for review)" and an empty
next step. We fill those in by hand.
## Do not build
Accounts, login, editing, deleting, voice input, sharing,
settings, styling beyond plain readable HTML.
## Tech
Python 3 standard library only (`http.server`, `csv`,
`json`, `urllib`), so it runs anywhere. Tests with
`unittest`. No AI call in the tests, use a stub.
## How we work
1. Write the plan as small steps in `PLAN.md` first. No
code before the plan.
2. One step at a time. Each step ends with a test that
passes.
3. After each passing step, save a version with a commit
that says what changed.
4. Never touch data in `thoughts.csv` that users entered.
Never commit the key.
```

A real build by an AI coding agent (Claude) from this brief, 19 October 2026, records in the course workspace. QuackSync is a made-up venture.



SHOW THE REAL THING · QUACKSYNC · DEMO 2 OF 6 · PLAN

Ask for a plan, not code

Each step small enough to test. No code before the plan.

PLAN.md as first committed · the steps, then step 1 in full

```
## Step 1: Save and list thoughts in a CSV file
## Step 2: Ask the AI model for a title and a next step
## Step 3: Save a thought, with the manual fallback
## Step 4: Render the page
## Step 5: Serve it over HTTP

## Step 1: Save and list thoughts in a CSV file
Adds: `thoughts.csv` storage. Check the input (not empty, at most 500 characters), append a thought with its title
and next step, read all thoughts back newest first.
Test that proves it: a saved thought comes back unchanged (commas, quotes and line breaks included), the newest
comes first, a missing file gives an empty list, an empty or 501-character input is refused and exactly 500 is
accepted.
```

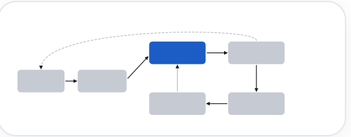
A real build by an AI coding agent (Claude) from this brief, 19 October 2026, records in the course workspace. QuackSync is a made-up venture.

NOW YOU · 1 MINUTE

Which step would you cut or split?

1. Save and list thoughts in a CSV file
2. Ask the AI model for a title and a next step
3. Save a thought, with the manual fallback
4. Render the page
5. Serve it over HTTP

1:00



SHOW THE REAL THING · QUACKSYNC · DEMO 3 OF 6 · ONE STEP

One step, one test, one version

Look at what changed. Files you didn't ask for? Ask why, or revert.

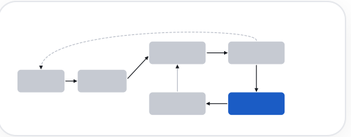
$ git show --stat · $ python3 -m unittest

```
$ git show --stat b7c65d0
b7c65d0 Step 3: add submit_thought, saves the model's title and next step or the '(waiting for review)' fallback

 quacksync.py      | 18 ++++++++++++++++
 test_quacksync.py | 64 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 82 insertions(+)

$ python3 -m unittest -v
...
Ran 25 tests in 0.005s

OK
```



SHOW THE REAL THING · QUACKSYNC · DEMO 4 OF 6 · REVIEW

The review caught what the tests didn't

Three reviews became three steps. The third finding didn't match the code, so the agent checked first.

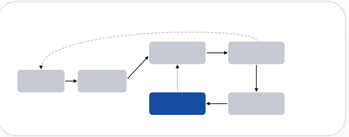
Review findings and fixes

```
## Step 6: Reject foreign Host and cross-site POSTs (review step)
## Step 7: Require a form token, fail closed (second review step)
## Step 8: Enforce localhost-only binding and say so in a README (third review step)
Checked before building: the finding does not match this code. The app reads one environment variable,
`ANTHROPIC_API_KEY`, and `main()` passes the literal `127.0.0.1` to the server. Started with `HOST=0.0.0.0
BIND=0.0.0.0`, the process still listens on `127.0.0.1` only (saved in `runs/step8-premise-check.txt`). No commit
ever read a `HOST` variable.

$ git log --oneline (review steps)
691e1d0 Reject foreign Host and cross-site POSTs (review finding)
584ff72 Require a form token, fail closed (second review)
905b57a Enforce localhost-only binding and add README (third review)

$ python3 -m unittest -v
...
Ran 59 tests in 11.146s
OK
```

A real security review of the build above, 19 October 2026. The fix is the build's own commit.



SHOW THE REAL THING · QUACKSYNC · DEMO 5 OF 6 · VERSIONS

Every step is a saved version

Two misses on a step? Go back to the last version and brief again.

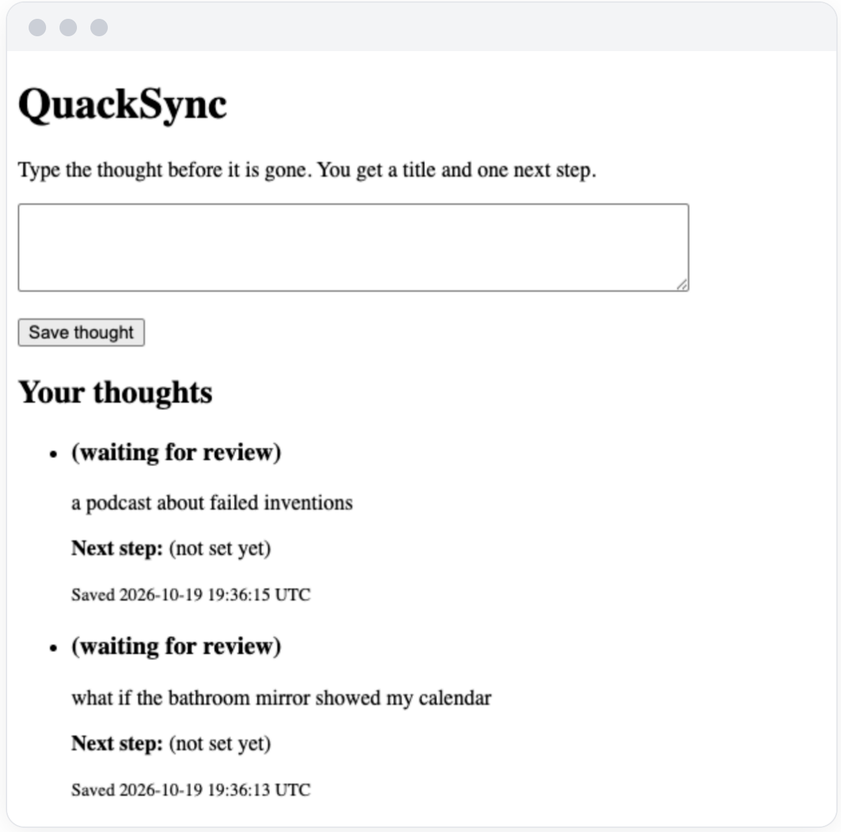
```
$ git log --oneline --reverse

3e276a0 Add project instructions
1a93994 Add plan in small steps
4fba017 Step 1: save thoughts to thoughts.csv and list them newest first, refuse empty or over-500-
character input
6503d72 Step 2: add ask_model, a Messages API call with urllib that returns a title and a next step,
tested with a stub
b7c65d0 Step 3: add submit_thought, saves the model's title and next step or the '(waiting for review)'
fallback
0ffc73f Step 4: add render_page, one plain HTML page with the form and the newest-first list, user text
escaped
4aa29a5 Step 5: add the local web server (GET / shows the page, POST /add saves and redirects) and the
command line start
8f44927 Add review step to plan: reject foreign Host and cross-site POSTs
691e1d0 Reject foreign Host and cross-site POSTs (review finding)
a34e1b2 Add second review step to plan: form token, fail closed
584ff72 Require a form token, fail closed (second review)
a75df3a Add third review step to plan: enforce localhost-only binding (finding's HOST premise not found
in code)
905b57a Enforce localhost-only binding and add README (third review)
```

A real build by an AI coding agent (Claude) from this brief, 19 October 2026, records in the course workspace. QuackSync is a made-up venture.

SHOW THE REAL THING · QUACKSYNC · DEMO 6 OF 6

No key yet, and it still works



1. Brief
2. Plan
3. One step
4. Review
5. Versions
6. **Result**

Both thoughts wait for review. The AI call was stubbed in the tests and never run. A person fills in the title.

A real build by an AI coding agent (Claude) from this brief, 19 October 2026, records in the course workspace. QuackSync is a made-up venture.

FROM YOUR LAPTOP TO THEIR HANDS

Live means a stranger can open it

Four things before you send the link

A link
opens on their phone

Its own data
kept apart, backed up

Keys in secrets
with a spending limit

Imprint, privacy
from your Module 04 page

On the hosting your tool brings, its publish button, a hosted sheet with a form

SPEED MATTERS

One iteration should not take more than

4 hours

Longer means the step was too big.

Cut it, or start over from your last version.

NOW YOU · ALONE

Write the first lines of your brief

Who — Who is it for?

Feature — The one feature, input and outcome

Manual — What stays by hand?

Not — What must it not build?

3:00

OUTPUT

Four lines your agent can read

CHAPTER 6 OF 9

Build to get users

Hand it to people from your segment, watch without helping, and fix what stopped them.

01 ~~Recap from last time~~
02 ~~Your presentations~~
03 ~~Every application ever~~
04 ~~The cheapest version that works~~
05 ~~You are the product owner~~
06 **Build to get users**
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

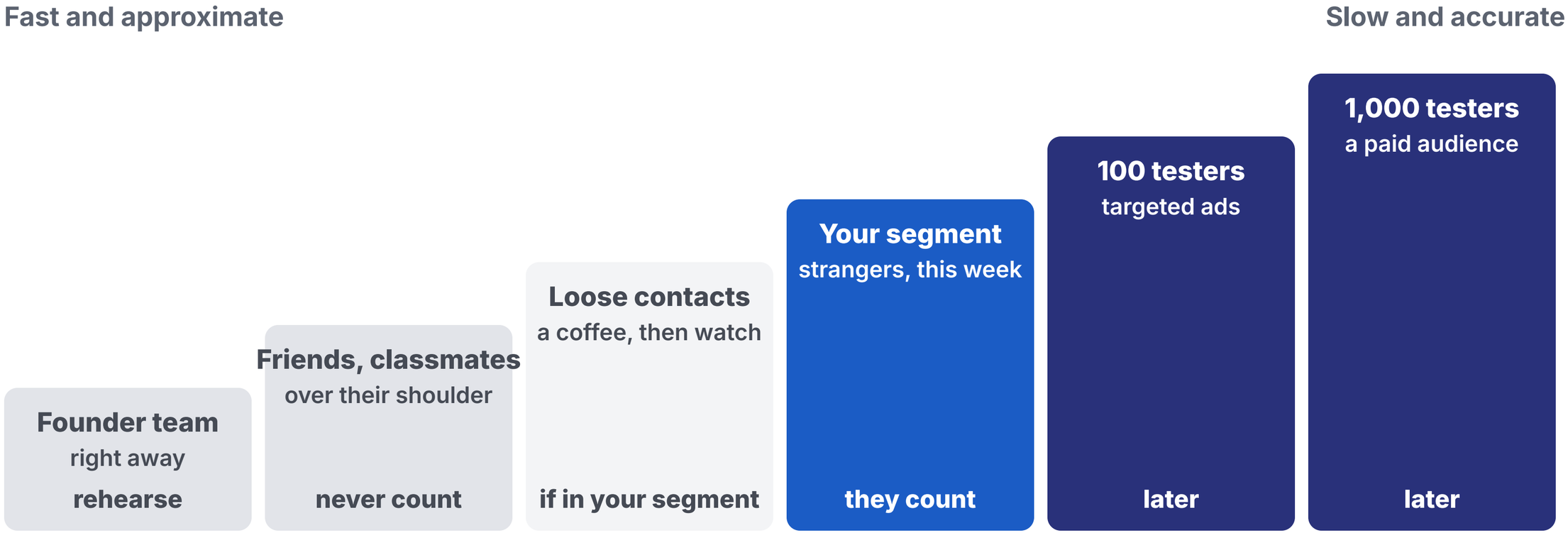
If you're not embarrassed by the first version of your product, you've launched too late.

Reid Hoffman, co-founder of LinkedIn

Hoffman, R. (2017, 29 March). If there aren't any typos in this essay, we launched too late! LinkedIn. Legacy slide 395.

TESTING YOUR PROTOTYPE

Friends rehearse. Your segment counts

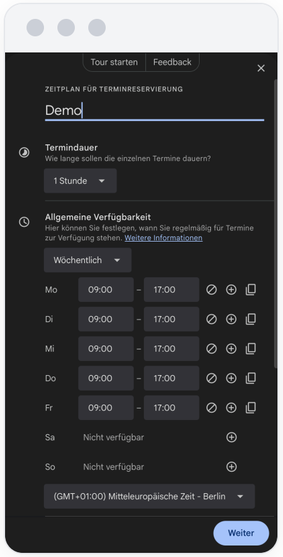


BASTIAN Testing ladder, legacy slide 237. Friends and classmates never count, M06 homework.

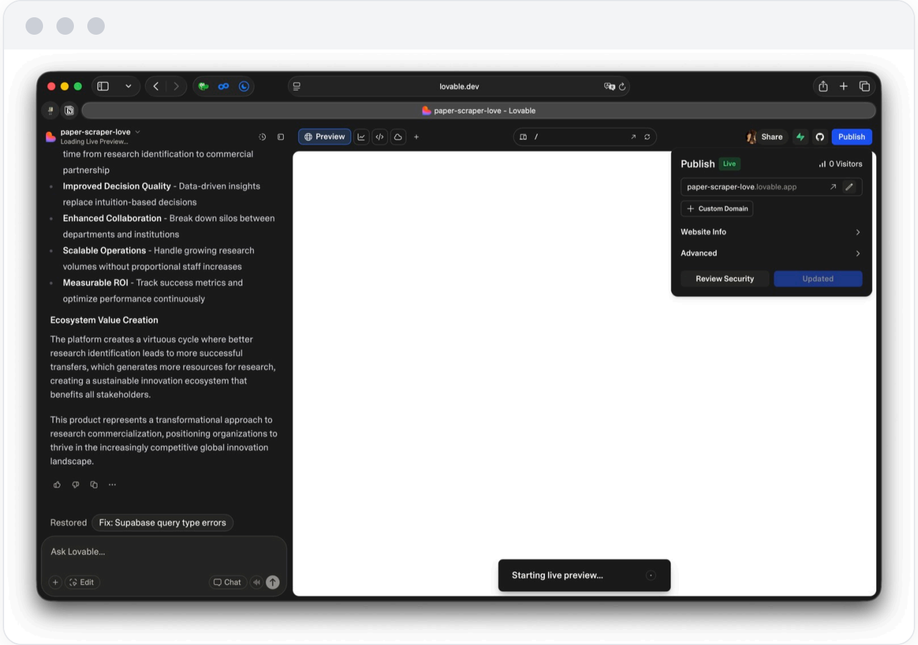
WITHOUT YOUR HELP

Let them use it alone

One hint and it is a guided demo again



Guided · Module 05
you book the call, you steer

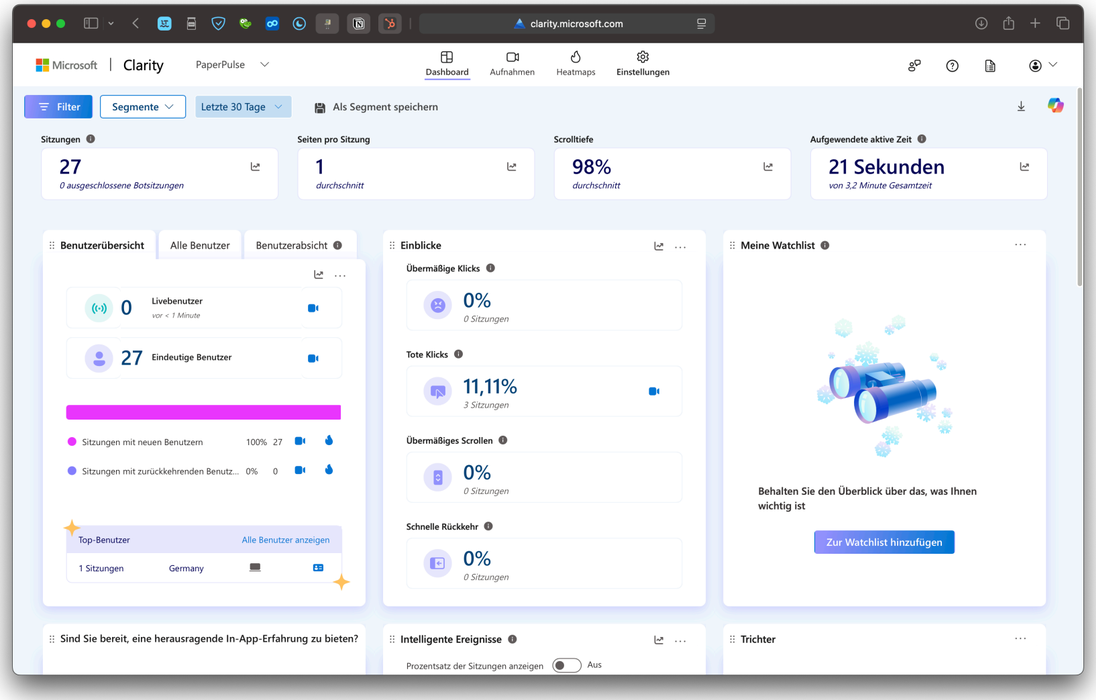


Without your help · this week
you publish one link, they decide what to do

WATCH WHAT THEY DO

Who came back, and where they got stuck

Returning users, dead clicks and a recording per session



Ask before you record. Consent covers course staff.

BASTIAN Microsoft Clarity on PaperPulse, Bastian's project, legacy slide 338 (German interface).

TWO QUESTIONS PER USER

Did they reach the outcome? Did they come back?

Reached the outcome?

the value arrived, without your help

Came back?

used it again, unprompted

User	Without help	Outcome	Came back	What stopped them
Recruiter, agency	yes	yes	day 3	–
Founder, B2B tool	yes	no	no	did not find "Add"

ILLUSTRATIVE Use record per user, made up. Framework 03 questions 4 and 8 read it in Module 07.

BEFORE EACH ITERATION

Four questions before you build again

1 **Does it match the problem?**
the Module 03 problem, in their words

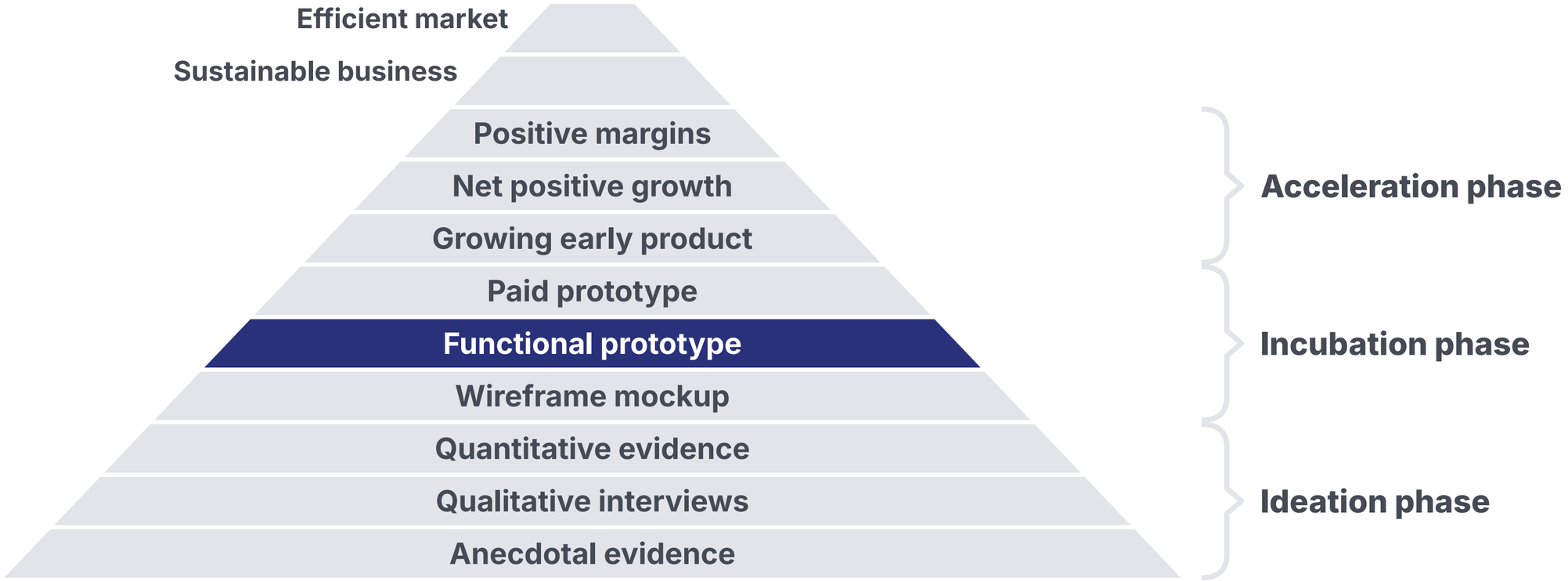
2 **For whom is it 10x better?**
than what they use today

3 **Can you build it in 4 hours?**
or is the step too big

4 **How does it reach their hands?**
a link, today, without you

WHERE YOU STAND

A slice that works is signal until someone pays



BASTIAN Hierarchy of evidence, legacy slides 128 and 328. Rungs 5 and 6, framework 02.

DON'T STRUGGLE ALONE

One leads the build. Everyone brings users

One founder leads the build

briefs the agent

one step at a time

ships the fixes

and brings users too

Everyone brings users

invites first buyers

watches, never steers

logs one row per user

NOW YOU · IN YOUR TEAM

Who gets the link today?

First Who accepted your Module 05 ask?

Then Which sign-ups and interviewees?

Watch How do you watch without steering?

2:00

OUTPUT

Five names and a way to watch

CHAPTER 7 OF 9

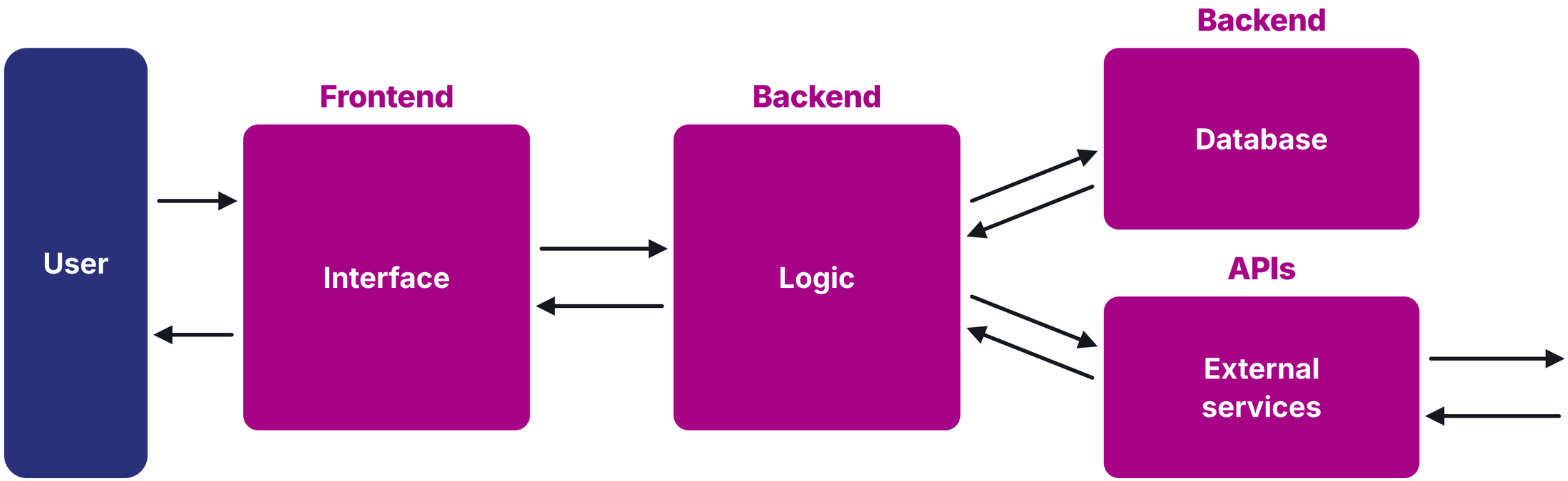
Wrap-up and check-out

Four parts, the cheapest version, the agent loop, and users without your help.

01 ~~Recap from last time~~
02 ~~Your presentations~~
03 ~~Every application ever~~
04 ~~The cheapest version that works~~
05 ~~You are the product owner~~
06 ~~Build to get users~~
07 **Wrap-up and check-out**
08 Workshop: build it, then hand it over
09 Case discussion

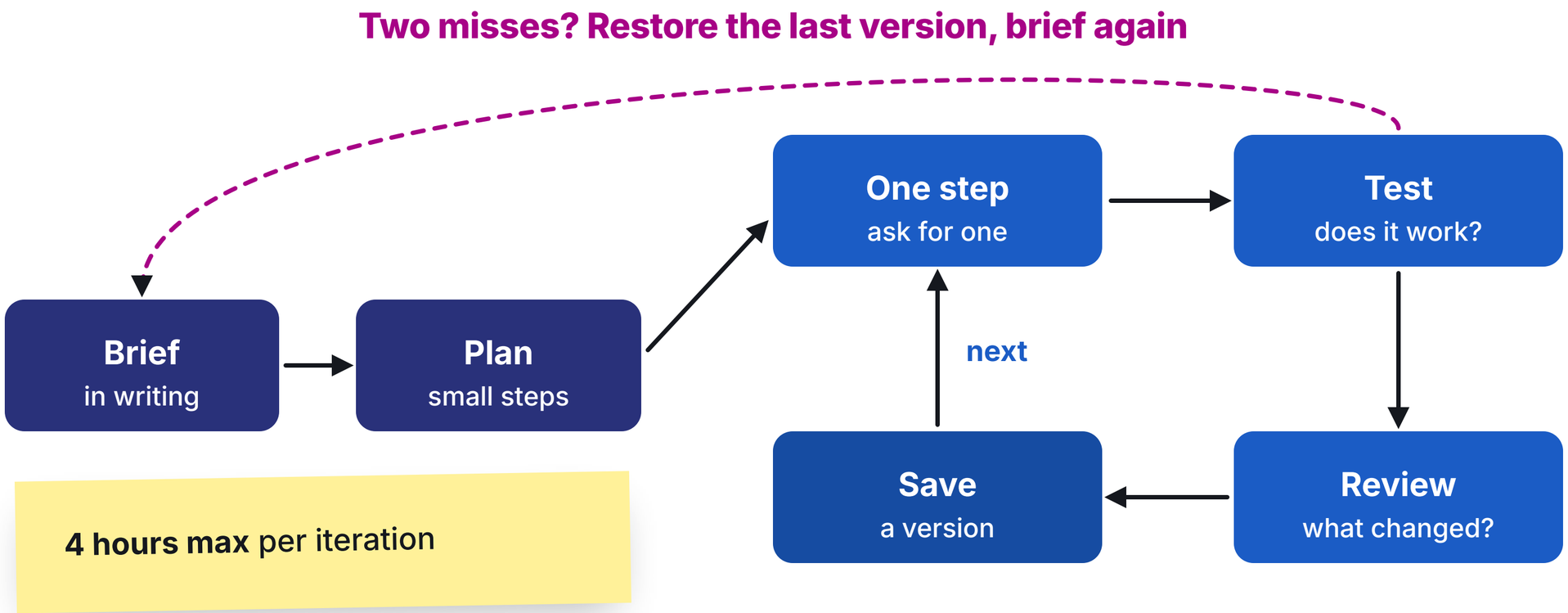
WRAP-UP · CHAPTER 3

Every application ever



WRAP-UP · CHAPTER 5

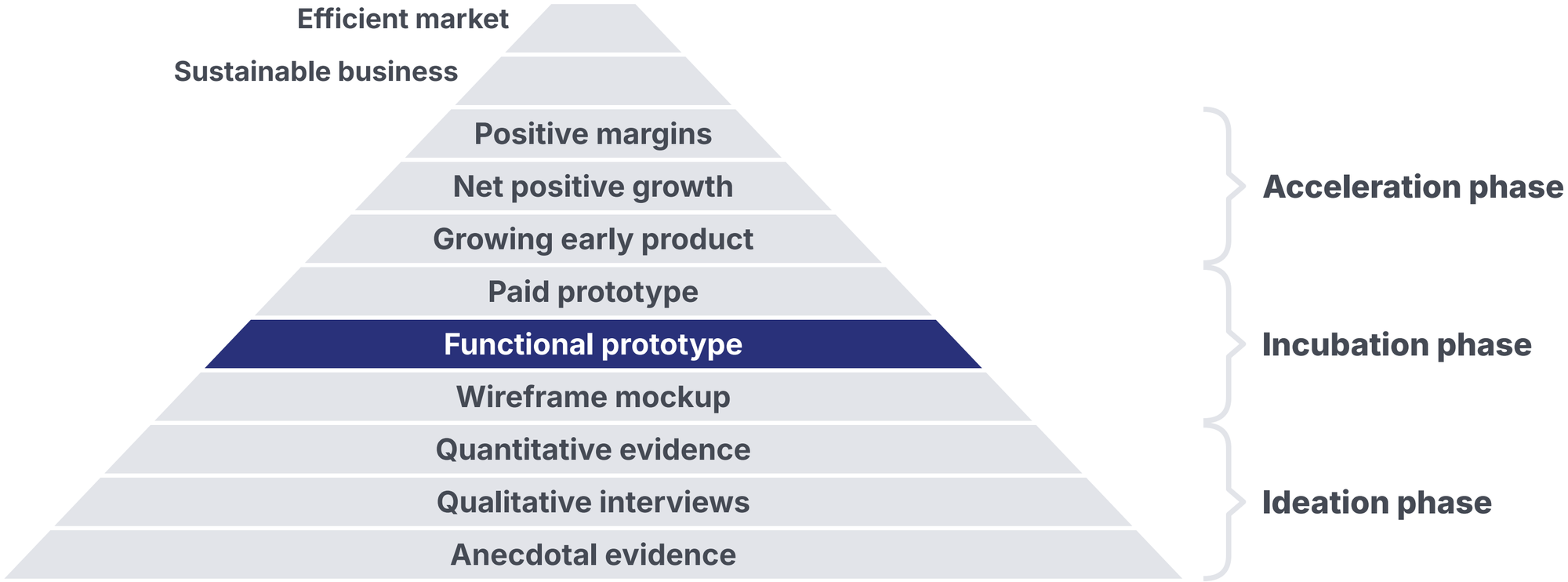
The agent loop



FRAMEWORK Claude Code docs, best practices, and the course resource Best Practices: Coding with AI.

WRAP-UP · CHAPTER 6

Rung five, signal until someone pays



REFLECTION 1 OF 3 · THE CONTENT

The Content – What could you apply



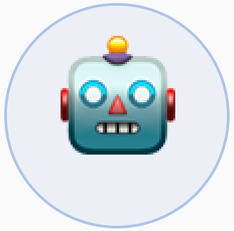
What could you apply?



What gave you a hard time?

REFLECTION 2 OF 3 · THE PROCESS

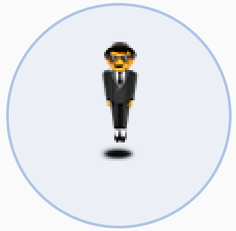
The Process – Man vs. Machine



Where did AI actually help you?



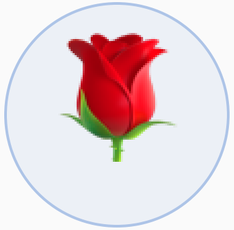
Where was your genius needed?



What are your best hacks to share?

REFLECTION 3 OF 3 · THE RESULTS

Rose, thorn, bud



What was the breakthrough today?



What was the hardest part today?



What is an idea/feature you didn't have time to finish, but want to explore later?

HOMEWORK UNTIL NEXT TIME

Put your MVP live, let 10 people per student from your segment use it without your help, and ship 2 fixes for what stopped them.

10 users each

The learning floor is a live MVP per team, 5 such users per student and 1 fix.

Your line: fewer than 3 of your first 10 users reach the outcome by Module 07, and you rebuild the feature. Nobody comes back unprompted, and you bring a pivot.

HOMEWORK · HOW TO DO IT

Hand it over. Watch. Fix what stopped them

1. **Split the work.** One leads the build, everyone brings users.
2. **Invite first buyers,** then sign-ups, then your segment's groups.
3. **Watch without steering.** Mute yourself or watch a recording.

! **Watch out.** One hint makes it a guided demo.

4. **Ask four questions before each build.** Does it match the problem? For whom is it 10x better? Can you build it in 4 hours? How does it reach them?
5. **Fix what stopped them, cut what nobody used.** Two misses? Restart from your last version.
6. **Keep their data.** Never wipe it. Collect only what you need, and say where it goes, AI included (Legal Basics).
7. **Keep the ask.** Show your price. An early payment counts.
8. **Get a blind second opinion.** Give an AI your raw records, minus hypothesis and names. Note where it disagrees.

HOMEWORK · WHAT YOU COME BACK WITH

Good is used, not clicked

	GOOD	WEAK
MVP	One feature, real input to outcome, 23 versions	Five features, "almost works"
USERS	31 recruiters without help, 14 got their nudge, 6 came back	12 friends clicked
FIXES	"Nobody found Add." Moved, 3 more reached it	Dark mode

ILLUSTRATIVE What you come back with, M06 homework. The examples follow the made-up recruiting venture.

REFERENCES

Sources in this deck

Best Practices: Coding with AI. Course resource, *Course/resources/Best_Practices_Coding_with_AI.pdf*.

Anthropic. (n.d.). *Best practices for Claude Code*. Claude Code Docs. https://code.claude.com/docs/en/best-practices, accessed 2026-10-19.

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). *Measuring the impact of early-2025 AI on experienced open-source developer productivity*. METR, 10 July 2025. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/, accessed 2026-10-19.

Hess, A. (2013, 14 August). *The oldest McDonald's as architecture*. Alan Hess on Architecture. https://alanhess.blogspot.com/2013/08/the-oldest-mcdonalds-as-architecture.html, accessed 2026-10-19.

Hoffman, R. (2017, 29 March). *If there aren't any typos in this essay, we launched too late!* LinkedIn. https://www.linkedin.com/pulse/arent-any-typos-essay-we-launched-too-late-reid-hoffman/, accessed 2026-10-19.

Ambitious Entrepreneurship. Framework 02, Validation Hierarchy, the Glossary and the course resource Best Practices: Coding with AI. Course files.

Graham, P. (2013, July). *Do things that don't scale*. paulgraham.com/ds.html.

See you next week!

MODULE 06 · PROTOTYPING WITH AI · WORKSHOP · 90 MINUTES

Workshop: Build it, then hand it over

Map the parts, build in small steps, and send the link before you leave.

01 Recap from last time
02 Your presentations
03 Every application ever
04 The cheapest version that works
05 You are the product owner
06 Build to get users
07 Wrap-up and check-out
08 **Workshop: build it, then hand it over**
09 Case discussion

WORKSHOP · STEP 1 OF 3

Map the parts

1 STEP 1 · 20 MIN

MAP THE PARTS

Know what must work.

→ Draw your MVP: interface, logic, database, outside services.
→ Mark each part: must work, or by hand for now.
→ Workflow or app? Would a form and a sheet do?
→ Write the brief: user, feature, input, outcome, manual, not-to-build.

Output A map and a brief.

Minutes 20

20:00

WORKSHOP · STEP 2 OF 3

Build in small steps

2 STEP 2 · 50 MIN

BUILD IN SMALL STEPS

Real input runs through to the outcome.

→ Load the brief as project instructions. Plan first, no code.
→ One step at a time: ask, read the change, test, save.
→ Two misses on one step? Restore and brief again.
→ Wire the one API call. Keep the key out of the interface.
→ Publish on your tool's hosting. Point your page's button at it.

Output A live link, versions saved.

Minutes 50

50:00

WORKSHOP · STEP 3 OF 3

Happy path, then hand it over

3 STEP 3 · 20 MIN

HAPPY PATH, THEN HAND IT OVER

First users today.

→ Log your line, dated, before the first user.
→ Run the happy path: real data in, watch it run, outcome out?
→ Send the link to your Module 05 first buyers. Don't steer.
→ Commit to the bar. Pick floor or full. Set a midweek day.

Output A dated line, users invited, targets, midweek day.

Minutes 20

20:00

ARRIVED EMPTY-HANDED?

Build the smallest slice that works

No winner? Build your best dummy's feature. Back at the problem? Build the smallest working slice, you behind the screen, and invite your interviewees.

HOMEWORK UNTIL NEXT TIME

Put your MVP live, let 10 people per student from your segment use it without your help, and ship 2 fixes for what stopped them.

10 users each

The learning floor is a live MVP per team, 5 such users per student and 1 fix.

Your line: fewer than 3 of your first 10 users reach the outcome by Module 07, and you rebuild the feature. Nobody comes back unprompted, and you bring a pivot.

See you next week!

MODULE 06 · PROTOTYPING WITH AI · CASE DISCUSSION · ABOUT 90 MINUTES

Case discussion: Langfuse

Stand with three founders deciding which borrowed part to replace first as strangers arrive.

01 Recap from last time
02 Your presentations
03 Every application ever
04 The cheapest version that works
05 You are the product owner
06 Build to get users
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 **Case discussion**

VOTE 1 · 16:00 UTC, 29 AUGUST 2023

What do the founders replace or build first?

1. A · Open analytics on an open-source tool for every user
2. B · Integrate OpenTelemetry
3. C · Model-based evaluations and labeling, their stated next step

2:00

TIMELINE · 2023

Fifteen weeks from first commit to strangers

DATE	EVENT
18 May	Repository created, "kill all", a fresh "init"
26 May	"v1"
1 June	Accounts and a hosted cloud
7 June	Sign-up by form, website button books a call with a founder
26 June	Public sign-up, "Get started by yourself" next to "Guided onboarding"

DATE	EVENT
17 July	Analytics on Looker Studio for flagged users
14 August	Demo project for every new user
20 August	Top product of the day on Product Hunt
29 August, 16:14 UTC	Show HN

DATA Exhibit 1 of the case, from the repositories, the founders' posts and Product Hunt. Langfuse case, M06, approved 2026-10-20.

EXHIBIT 3, SELECTED ROWS

What a new user met on 29 August 2023

WHAT	STATE ON 29 AUGUST 2023	SINCE
Start screen	"Create new project" (get started by yourself), "View demo project" and "Guided onboarding" (a call with a founder)	26 June, as on 27 August
Demo project	View-only, live data from the documentation chatbot	14 August
Dashboards	Traces, token usage by model, scores, per-user views, for every user	By 27 August (video)
Analytics (Looker Studio)	For users with the flag "analytics-alpha" only. Others saw "Analytics is in closed alpha" and an invitation to get in touch	17 July
Support	Feedback button, support chat offered while setting up a project, a Discord community	23 June, 29 July
Account	No way to change or reset the password	Open issue of 21 August

DATA Exhibit 3 of the Langfuse case, selected rows, with its source line in the case.

IN THE FOUNDERS' WORDS · 29 AUGUST 2023

Half a day, or a debt?

"Integrating Looker Studio got us to first analytics charts in half a day."

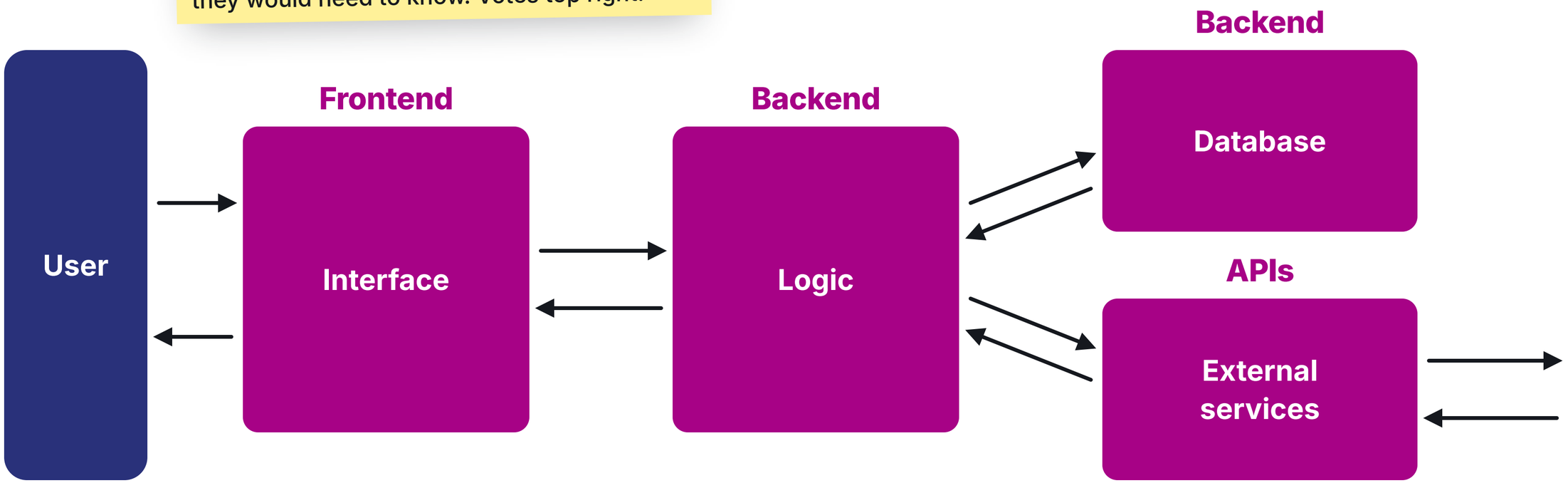
"As it is not open-source and does not work with our UI/UX, we are looking to switch it out for an OSS solution to flexibly generate charts and dashboards."

DATA Marc Klingen, Show HN post, 29 August 2023, as quoted in the case (Q06, Q07). Langfuse case, M06, approved 2026-10-20.

BOARD PLAN

Mark what still needs a founder

Columns: borrowed and why · what it costs a stranger · what they knew about use · what they would need to know. Votes top right.



BASTIAN Board plan of the Langfuse teaching note. Every application ever, legacy slides 398 and 400 to 402.

USE WITHOUT HELP · ON 29 AUGUST 2023

What did they know about use?

One user, many hours a day
as Deichmann reported it

Five named teams
on the landing page

Alpha users onboarded every week
the July update

370 stars, 21 forks
fifteen minutes after the post

No published number for sign-ups, active users or returning users.

VOTE 2 · SAME QUESTION

What do the founders replace or build first?

1. A · Open analytics on an open-source tool for every user
2. B · Integrate OpenTelemetry
3. C · Model-based evaluations and labeling, their stated next step

2:00

BRIDGE · TO YOUR OWN BUILD

Mark what still needs you

Langfuse let strangers in with a call, a chat and a flag still in place. Now draw your own MVP on the four parts and mark what still needs you.

See you next week!